

FACULDADE DE TECNOLOGIA PREFEITO HIRANT SANAZAR

DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

BANCO DE DADOS RELACIONAL

CAROLINA DE OLIVEIRA ALVES

**DESENVOLVIMENTO DE UM SISTEMA DE RECURSOS HUMANOS
UTILIZANDO BANCO DE DADOS RELACIONAL**

Osasco

2026

DESENVOLVIMENTO DE UM SISTEMA DE RECURSOS HUMANOS UTILIZANDO BANCO DE DADOS RELACIONAL

Trabalho apresentado no curso de
Desenvolvimento de Software
Multiplataforma da Faculdade de
Tecnologia Prefeito Hiranr Sanazar.

Professor: Hélio Oliveira

Osasco
2026

SUMÁRIO

1 INTRODUÇÃO.....	4
2 FUNDAMENTAÇÃO TEÓRICA.....	5
2.1 Subconsultas.....	5
2.1.1 Atividade Conceitual.....	6
2.2 CTE — Common Table Expression.....	6
2.2.1 Atividade Conceitual.....	7
2.3 EXISTS / IN / ANY / ALL.....	8
2.3.1 Atividade Conceitual.....	9
2.4 VIEWS.....	9
2.4.1 Atividade Conceitual.....	10
2.5 Programação no SGBD.....	11
2.5.1 Atividade Conceitual.....	12
2.6 Análise Técnica.....	13
2.6.1 EXISTS vs IN.....	13
2.6.2 CTE Recursiva.....	13
2.6.3 Riscos do uso incorreto de Triggers.....	14
2.6.4 Vantagens da programação no SGBD.....	14
2.6.5 VIEWS e segurança.....	14
3 DESENVOLVIMENTO PRÁTICO.....	15
3.1. Subconsulta — Análise de Funcionários Acima da Média Salarial.....	15
3.2. CTE — Relatório Hierárquico de Gestores.....	15
3.3. EXISTS — Verificação de Funcionários com Benefícios Ativos.....	16
3.4. IN — Filtro de Funcionários por Nível de Cargo.....	17
3.5. ANY — Comparação Salarial com a Faixa Júnior.....	18
3.6. ALL — Funcionários Completamente Acima da Faixa Júnior.....	18
3.7. VIEW — Indicadores Gerenciais por Departamento.....	19
3.8. Procedure — Processamento da Folha de Pagamento Mensal.....	20
3.9. Function — Cálculo do Salário Líquido.....	20
3.10. Trigger AFTER — Auditoria de Alterações Salariais.....	21
3.11. Trigger INSTEAD OF — Bloqueio de Exclusão Física.....	22
4 ESTUDO DE CASO.....	23
5 CONCLUSÃO.....	25
6 REFERÊNCIAS BIBLIOGRÁFICAS.....	26

1 INTRODUÇÃO

A administração de departamentos de Recursos Humanos envolve a manipulação constante de grandes volumes de dados sensíveis, que vão desde o registro diário de frequência até o cálculo detalhado de encargos e folhas de pagamento. Em organizações de médio e grande porte, falhas na integridade dessas informações ou atrasos no processamento podem gerar graves passivos trabalhistas e prejuízos operacionais. Por essa razão, a automação baseada em regras de negócio implementadas diretamente na camada do banco de dados tornou-se uma prática indispensável para garantir a consistência física e lógica dos dados relacionados aos colaboradores de uma empresa.

Este trabalho tem como objetivo contextualizar a utilização de um banco de dados relacional no setor de Recursos Humanos de uma empresa, demonstrando como os conceitos e recursos dos SGBDs podem ser aplicados para melhorar o gerenciamento das informações dos colaboradores, aumentar a confiabilidade dos dados e apoiar as atividades operacionais e administrativas da organização.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Subconsultas

Uma subconsulta, também chamada de subquery, é uma consulta SQL escrita dentro de outra consulta. Ela é executada primeiro e seu resultado é utilizado pela consulta externa para filtrar, calcular ou exibir dados. No sistema de RH, por exemplo, é possível usar uma subconsulta para calcular a média salarial da empresa e, a partir desse resultado, listar apenas os funcionários que recebem acima dela, sem precisar executar duas consultas separadas.

O funcionamento ocorre em duas etapas: o banco de dados executa a subconsulta interna, obtém o resultado, e então utiliza esse valor como critério ou dado para a consulta externa. Dependendo de onde a subconsulta é escrita e de como ela se relaciona com a consulta externa, ela pode ser classificada em diferentes tipos.

A subconsulta simples é executada apenas uma vez e seu resultado é reaproveitado pela consulta externa. No sistema de RH, calcular a média salarial uma única vez e comparar com cada funcionário é um exemplo desse tipo. A subconsulta correlacionada, por sua vez, referencia colunas da consulta externa e é reexecutada para cada linha processada. No sistema de RH, buscar o maior salário dentro do departamento de cada funcionário exige uma subconsulta correlacionada, pois o departamento muda a cada linha avaliada.

A subconsulta no SELECT retorna um valor escalar exibido como coluna adicional no resultado, como mostrar a nota da última avaliação de desempenho ao lado dos dados de cada funcionário. A subconsulta no FROM cria uma tabela temporária chamada de derived table, utilizada como fonte de dados pela consulta externa, sendo útil para pré-agregar informações antes de fazer joins. A subconsulta no WHERE é a mais comum e serve para filtrar registros com base em valores calculados dinamicamente, como listar funcionários de departamentos com mais de três colaboradores ativos.

Entre as vantagens das subconsultas estão a clareza na separação da lógica, a possibilidade de resolver problemas em uma única instrução SQL e a flexibilidade para consultas dinâmicas que dependem de valores calculados em tempo de execução. Como desvantagens, subconsultas correlacionadas podem ter desempenho inferior em tabelas grandes, pois são reexecutadas para cada linha, e consultas muito aninhadas tendem a dificultar a leitura e manutenção do código.

2.1.1 Atividade Conceitual

O que é uma subconsulta? É uma consulta com SELECT dentro de outra consulta SELECT. Primeiro a subconsulta é executada e seu resultado é usado pela consulta externa. No sistema de RH, uma subconsulta pode ser utilizada para encontrar funcionários que recebem salário acima da média da empresa.

Diferença entre subconsulta simples e correlacionada A simples funciona de forma independente. Já a correlacionada precisa dos dados da consulta principal para gerar o resultado, podendo verificar se cada funcionário possui registros de ponto associados.

2.2 CTE — Common Table Expression

Uma CTE, definida pela cláusula WITH, é um recurso que permite nomear e reutilizar o resultado de uma consulta dentro do escopo de uma instrução SQL. Diferente de uma subconsulta no FROM, a CTE é declarada antes da consulta principal e pode ser referenciada múltiplas vezes, tornando o código mais organizado e legível. No sistema de RH, em vez de repetir o cálculo da média salarial em vários pontos da consulta, é possível declará-lo uma única vez em uma CTE e referenciá-lo sempre que necessário.

A sintaxe básica de uma CTE é composta pela palavra-chave WITH seguida do nome da CTE, da definição da consulta entre parênteses e da consulta principal que a utiliza. É possível declarar múltiplas CTEs separadas por vírgula antes da consulta

principal, o que permite encadear transformações de dados de forma progressiva e clara.

A CTE recursiva é um tipo especial que referencia a si mesma e é utilizada para percorrer estruturas hierárquicas ou em árvore. Ela é composta por dois blocos unidos por UNION ALL: o membro âncora, que define o ponto de partida da recursão, e o membro recursivo, que se une à própria CTE para avançar nível a nível. No sistema de RH, a hierarquia de gestores é armazenada na tabela funcionario por meio do campo id_gestor, e a CTE recursiva permite montar o organograma completo da empresa partindo dos diretores e descendo até os cargos operacionais.

Em termos de performance, as CTEs no SQL Server são expandidas internamente pelo otimizador de consultas e não são materializadas por padrão, o que significa que podem ser reexecutadas a cada referência. Para volumes muito grandes de dados, subconsultas ou tabelas temporárias podem ter desempenho superior. A principal diferença entre CTE e VIEW é o escopo: a CTE existe apenas durante a execução da instrução em que foi declarada, enquanto a VIEW é um objeto persistente no banco que pode ser consultado por qualquer instrução a qualquer momento.

2.2.1 Atividade Conceitual

O que é uma CTE? Uma estrutura temporária dentro da consulta juntamente com WITH para organizar e simplificar os dados, usada para calcular previamente as horas trabalhadas dos funcionários antes de gerar um relatório da folha de pagamento.

Vantagem do WITH Ela filtra e define as CTEs, deixando as consultas mais fáceis de entender e manter, assim organizando cálculos de horas extras e salários em etapas separadas dentro da mesma consulta.

2.3 EXISTS / IN / ANY / ALL

EXISTS, IN, ANY e ALL são operadores utilizados em subconsultas para comparar ou verificar a existência de valores. Cada um possui comportamento e casos de uso específicos que impactam diretamente na legibilidade e no desempenho das consultas.

O operador EXISTS verifica se uma subconsulta retorna ao menos uma linha. Ele não se importa com os valores retornados, apenas com a existência de resultado, por isso é comum utilizar SELECT 1 dentro dele por convenção. Assim que a primeira linha é encontrada, a execução é interrompida, o que torna o EXISTS eficiente em tabelas com grande volume de dados. No sistema de RH, EXISTS é utilizado para verificar se um funcionário possui avaliação de desempenho registrada ou se possui benefícios ativos, sem precisar trazer os dados desses registros para a consulta principal.

O operador IN compara um valor com uma lista de resultados retornados por uma subconsulta ou informados diretamente. Diferente do EXISTS, o IN materializa toda a lista antes de fazer as comparações, o que pode ser menos eficiente quando a subconsulta retorna muitos registros. No sistema de RH, o IN é utilizado para filtrar funcionários por nível de cargo, como Senior e Pleno, ou para listar apenas colaboradores de departamentos com equipe mínima formada.

A principal diferença entre EXISTS e IN está no comportamento interno: o EXISTS opera com curto-circuito e é mais eficiente quando a subconsulta retorna muitas linhas ou quando basta verificar a existência. O IN é mais legível para listas pequenas e fixas, mas pode ter desempenho inferior em volumes maiores. Em relação ao EXISTS versus JOIN, o JOIN traz os dados relacionados para o resultado enquanto o EXISTS apenas verifica a existência sem trazer dados adicionais, sendo preferível quando o objetivo é apenas filtrar registros.

O operador ANY retorna verdadeiro se a comparação for satisfeita por ao menos um valor da subconsulta, funcionando de forma equivalente ao SOME. No sistema de RH, ANY é utilizado para identificar funcionários cujo salário supera ao menos um

salário da faixa júnior, evidenciando sobreposição salarial entre níveis de cargo. O operador ALL retorna verdadeiro apenas se a comparação for satisfeita por todos os valores da subconsulta. No sistema de RH, ALL é usado para encontrar funcionários cujo salário está completamente acima de toda a faixa salarial júnior, identificando quem está além do teto dessa categoria.

2.3.1 Atividade Conceitual

Quando utilizar EXISTS? EXISTS verifica se uma subconsulta retorna pelo menos um registro existente, como por exemplo, verificar quais funcionários possuem avaliações de desempenho cadastradas.

Diferença entre EXISTS e IN IN procura valores específicos como listar funcionários de determinados departamentos. EXISTS apenas confirma se há registros relacionados, como verificar se um funcionário possui registros de ponto.

Função do ANY ANY compara um valor com qualquer valor retornado por uma subconsulta, ou seja, a condição será verdadeira se pelo menos um valor da subconsulta atender ao critério, como encontrar funcionários com salário maior que pelo menos um dos salários de um determinado departamento.

Função do ALL ALL compara um valor com todos os valores retornados de uma subconsulta, onde só será verdadeira se atender à comparação com todos os resultados, como identificar funcionários cujo salário é maior que todos os salários de outro departamento.

2.4 VIEWS

Uma VIEW é um objeto do banco de dados que armazena uma consulta SQL nomeada e permite que ela seja utilizada como se fosse uma tabela. Os dados não são duplicados: cada vez que a VIEW é consultada, a instrução SQL original é executada e o resultado é retornado dinamicamente. No sistema de RH, as VIEWS

são utilizadas para criar camadas de acesso aos dados, simplificar consultas complexas e proteger informações sensíveis.

O objetivo principal das VIEWS é a abstração e a reutilização. Em vez de cada desenvolvedor ou analista precisar conhecer toda a estrutura de joins e filtros do banco, eles consultam a VIEW que já encapsula essa complexidade. A VIEW `vw_funcionarios_completo` do sistema de RH, por exemplo, já reúne dados do funcionário, departamento, cargo e gestor em uma única consulta pronta para uso.

Em relação aos tipos, a VIEW simples é baseada em uma única tabela sem agregações e pode ser atualizável, ou seja, permite executar INSERT, UPDATE e DELETE diretamente sobre ela, refletindo as alterações na tabela original. A VIEW complexa envolve múltiplas tabelas com joins, e em geral não é atualizável. A VIEW com agregação utiliza funções como SUM, AVG e COUNT para consolidar dados, como a `vw_indicadores_departamento` que calcula a folha salarial e a média de avaliação por departamento.

A segurança é uma das principais vantagens das VIEWS. É possível conceder acesso a uma VIEW sem conceder acesso direto às tabelas subjacentes, controlando exatamente quais colunas e linhas cada usuário pode visualizar. No sistema de RH, a `vw_funcionarios_publico` expõe apenas nome, email e telefone, ocultando salário, CPF e data de nascimento para usuários de outros departamentos que não precisam dessas informações. Além disso, as VIEWS facilitam a reutilização pois centralizam a lógica de consulta em um único ponto, garantindo que qualquer alteração na regra seja refletida automaticamente para todos que utilizam a VIEW.

2.4.1 Atividade Conceitual

O que é uma VIEW? VIEW é uma tabela virtual criada a partir de uma consulta já existente. Podemos criar uma view para exibir funcionários, cargos e departamentos em uma única consulta.

Vantagens das VIEWS As views simplificam consultas, aumentam a segurança e promovem a reutilização de código, evitando repetir consultas complexas. Podem

limitar o acesso a determinados dados, como fornecer aos gestores apenas informações necessárias dos colaboradores, sem expor dados sensíveis.

2.5 Programação no SGBD

A programação no SGBD consiste em armazenar lógica de negócio diretamente no banco de dados por meio de objetos programáveis como stored procedures, functions e triggers. Essa abordagem centraliza as regras no banco, garantindo que sejam aplicadas independentemente da linguagem ou sistema que acessa os dados. No sistema de RH, o cálculo da folha de pagamento, o registro de ponto e o reajuste salarial são exemplos de processos automatizados diretamente no banco.

As vantagens da programação no SGBD incluem desempenho, pois o código é executado próximo aos dados eliminando tráfego de rede; segurança, pois é possível conceder permissão apenas para executar a procedure sem expor as tabelas diretamente; reusabilidade, pois a lógica é escrita uma vez e reutilizada por qualquer sistema; e consistência, pois regras como o cálculo de INSS e FGTS são aplicadas de forma idêntica em toda e qualquer operação que acione a procedure.

As stored procedures são blocos de código SQL nomeados que aceitam parâmetros de entrada e saída, podendo conter variáveis, estruturas condicionais, laços de repetição, transações e tratamento de erros. No sistema de RH, a `sp_processar_folha_mensal` utiliza um cursor para percorrer todos os funcionários ativos, calculando para cada um os descontos e inserindo o registro na folha de pagamento dentro de uma transação que garante que ou todos os registros são inseridos com sucesso ou nenhum é, preservando a integridade dos dados. O tratamento de erros com TRY/CATCH captura qualquer falha e realiza o rollback automaticamente.

As functions são semelhantes às procedures, mas são obrigadas a retornar um valor e não podem produzir efeitos colaterais como INSERT, UPDATE ou DELETE. Existem dois tipos principais: a scalar function, que retorna um único valor e pode ser usada diretamente em um SELECT, e a table-valued function, que retorna uma tabela e pode ser usada como fonte de dados em um FROM. No sistema de RH, a

fn_calcular_salario_liquido é uma scalar function que recebe o id do funcionário e retorna o salário líquido após todos os descontos, enquanto a fn_relatorio_ponto_funcionario é uma table-valued function que retorna o espelho de ponto de um período com o status de cada dia.

As triggers são procedimentos executados automaticamente pelo banco em resposta a eventos DML como INSERT, UPDATE ou DELETE em uma tabela. A trigger AFTER é disparada após a operação ser concluída e é utilizada principalmente para auditoria e histórico, como a trg_auditoria_salario que registra no log toda alteração de salário com o valor anterior, o valor novo, a data e o usuário responsável. A trigger INSTEAD OF substitui a operação original por uma lógica personalizada, como a trg_bloquear_exclusao_funcionario que impede a exclusão física de funcionários com histórico no sistema, realizando em seu lugar a desativação lógica via data_demissao e registrando a tentativa no log.

Os riscos do uso incorreto de triggers incluem loops infinitos quando a trigger modifica a própria tabela que a disparou, impacto no desempenho quando lógicas pesadas são executadas a cada operação DML, dificuldade de manutenção pois o comportamento automático pode ser esquecido por desenvolvedores que não conhecem as triggers cadastradas, e rollback involuntário de transações legítimas quando um erro dentro da trigger desfaz toda a operação original.

2.5.1 Atividade Conceitual

O que é uma Stored Procedure? Stored Procedure é um conjunto de comandos SQL armazenados no banco de dados e executados sob demanda, sendo uma rotina pronta que automatiza tarefas repetitivas. Pode ser usada para automatizar o cálculo e a geração da folha de pagamento mensal.

Diferença entre Procedure e Function Procedures fazem processos completos, executam ações e podem retornar vários resultados, como gerar a folha de pagamento. Functions obrigatoriamente retornam um valor, podendo calcular o valor das horas extras de um funcionário.

O que é uma Trigger? Trigger é uma ação executada automaticamente quando ocorre um evento no banco de dados, como INSERT, UPDATE ou DELETE. Podemos ver uma trigger no processo de registrar automaticamente alterações de salário em uma tabela de auditoria.

Diferença entre AFTER e INSTEAD OF AFTER executa a trigger após a operação ser concluída, como registrar alterações de funcionários em auditoria. INSTEAD OF substitui a operação original, podendo impedir a exclusão de registros importantes.

O que é auditoria em banco de dados? Auditoria funciona como um histórico que mostra quem alterou, quando alterou e o que foi alterado. No sistema de RH, a auditoria pode registrar mudanças em salários, cargos e avaliações de desempenho, garantindo rastreabilidade e segurança das informações.

2.6 Análise Técnica

2.6.1 EXISTS vs IN

O EXISTS é mais eficiente que o IN quando a subconsulta retorna muitos registros, pois ele interrompe a busca assim que encontra a primeira ocorrência. No sistema de RH, ao buscar funcionários que possuem benefícios ativos, o EXISTS verifica apenas a existência do registro na tabela beneficio e para imediatamente, enquanto o IN precisaria carregar todos os ids de beneficiários para depois comparar com cada funcionário.

2.6.2 CTE Recursiva

A CTE recursiva é indicada quando os dados possuem relacionamento hierárquico pai-filho dentro da mesma tabela. No sistema de RH, a tabela funcionario possui o campo id_gestor que referencia o próprio id_funcionario, formando uma hierarquia de gestores. A CTE recursiva percorre essa estrutura partindo dos diretores e descendo nível a nível até os cargos mais baixos, montando o organograma completo da empresa.

2.6.3 Riscos do uso incorreto de Triggers

Triggers executam automaticamente a cada operação DML e podem causar problemas sérios se mal implementadas. No sistema de RH, uma trigger mal escrita na tabela `funcionario` poderia ser disparada a cada update de salário, inserindo registros duplicados no log ou causando loops infinitos caso a própria trigger modifique a tabela que a disparou. Além disso, erros dentro de uma trigger desfazem toda a transação original, podendo impedir atualizações legítimas.

2.6.4 Vantagens da programação no SGBD

Centralizar a lógica no banco garante que as regras de negócio sejam aplicadas independentemente da aplicação que acessa os dados. No sistema de RH, a procedure `sp_processar_folha_mensal` calcula descontos de INSS, FGTS e benefícios diretamente no banco, garantindo que qualquer sistema que acesse o RH sempre use o mesmo cálculo, sem risco de divergência entre versões de aplicações diferentes.

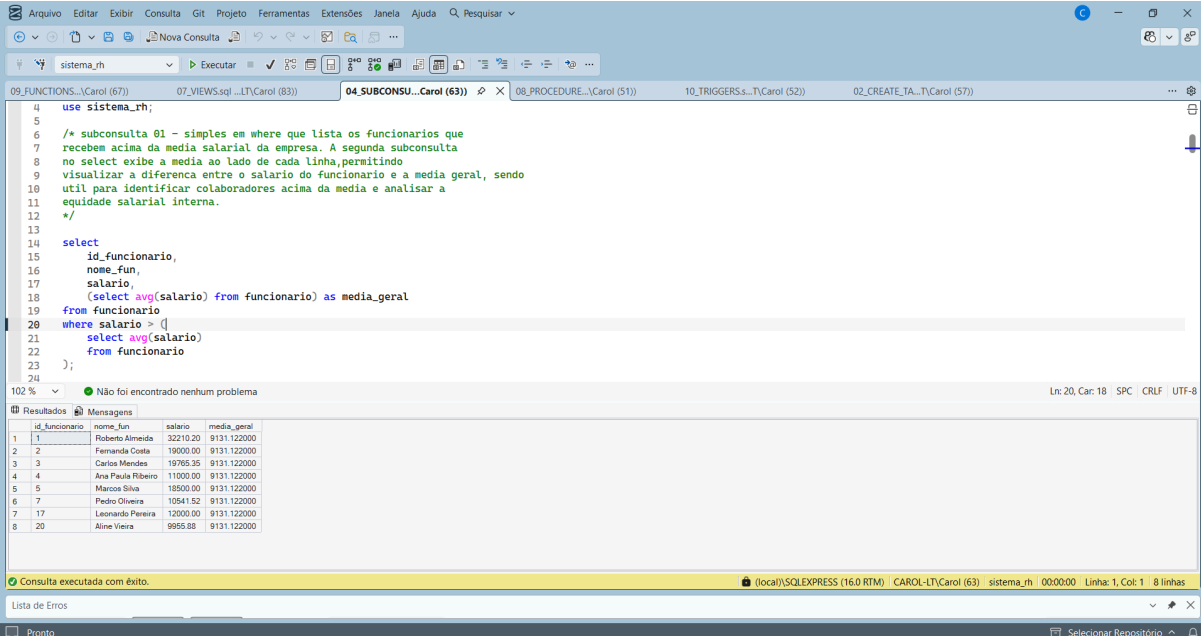
2.6.5 VIEWS e segurança

As views permitem expor apenas os dados necessários para cada perfil de usuário, ocultando colunas sensíveis. No sistema de RH, a `vw_funcionarios_publico` expõe apenas nome, email e telefone, ocultando salário e CPF. Isso permite conceder acesso à view para usuários de outros departamentos sem que eles consigam visualizar informações confidenciais da tabela `funcionario` diretamente.

3 DESENVOLVIMENTO PRÁTICO

3.1. Subconsulta — Análise de Funcionários Acima da Média Salarial

Para identificar os colaboradores com remuneração acima da média da empresa, foi utilizada uma subconsulta simples no WHERE que calcula a média salarial de todos os funcionários ativos, e uma segunda subconsulta escalar no SELECT que exibe essa média ao lado de cada linha, permitindo visualizar a diferença entre o salário individual e a média geral. O resultado retornou 8 funcionários acima da média de R\$ 9.131,12, liderados por Roberto Almeida com salário de R\$ 32.210,20.



The screenshot shows a SQL IDE window with a query editor and a results pane. The query is as follows:

```
4 use sistema_rh;
5
6 /* subconsulta 01 - simples em where que lista os funcionarios que
7 recebem acima da media salarial da empresa. A segunda subconsulta
8 no select exibe a media ao lado de cada linha, permitindo
9 visualizar a diferenca entre o salario do funcionario e a media geral, sendo
10 util para identificar colaboradores acima da media e analisar a
11 equidade salarial interna.
12 */
13
14 select
15     id_funcionario,
16     nome_fun,
17     salario,
18     (select avg(salario) from funcionario) as media_geral
19 from funcionario
20 where salario > (
21     select avg(salario)
22     from funcionario
23 );
24
```

The results pane shows the following data:

id_funcionario	nome_fun	salario	media_geral
1	Roberto Almeida	32210.20	9131.122000
2	Fernanda Costa	19000.00	9131.122000
3	Carlos Mendes	19765.35	9131.122000
4	Ana Paula Ribeiro	11000.00	9131.122000
5	Marcos Silva	18500.00	9131.122000
6	Pedro Oliveira	10541.52	9131.122000
7	Leonardo Pereira	12000.00	9131.122000
8	Aline Vieira	9955.88	9131.122000

The status bar at the bottom indicates: "Consulta executada com êxito." (Query executed successfully.)

3.2. CTE — Relatório Hierárquico de Gestores

Para mapear a estrutura organizacional da empresa, foi utilizada uma CTE recursiva que parte dos funcionários sem gestor, identificados como nível 0, e desce nível a nível até os cargos operacionais. O resultado exibe o organograma completo com o caminho hierárquico de cada colaborador, como "Fernanda Costa > Ana Paula Ribeiro > Beatriz Santos", permitindo visualizar toda a cadeia de gestão do sistema de RH com 24 funcionários mapeados.

The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays a SQL query in the '05.CTE.sql' file. The query is a recursive CTE named 'hierarquia_rh' that calculates the organizational hierarchy, including employee names, salaries, and a path ('caminho') from the root to each employee. The bottom pane shows the results of the query, which are 14 rows of data. The results are displayed in a table with columns: organograma, nivel, salario, and caminho. The status bar at the bottom indicates that the query was executed successfully.

```
188 de cada funcionario na estrutura organizacional e o caminho completo ate a diretoria. */
189
190 with hierarquia_rh as (
191     select
192         f.id_funcionario,
193         f.nome_fun,
194         f.salario,
195         f.id_gestor,
196         0 as nivel,
197         cast(f.nome_fun as varchar(500)) as caminho
198     from funcionario f
199     where f.id_gestor is null
200     and f.data_demissao is null
201     union all
202     select
203         f.id_funcionario,
204         f.nome_fun,
205         f.salario,
206         f.id_gestor,
207         h.nivel + 1,
208         cast(h.caminho + ' > ' + f.nome_fun as varchar(500))
209     from funcionario f
210     inner join hierarquia_rh h on h.id_funcionario = f.id_gestor
211     where f.data_demissao is null
212 )
213 select
214     replace(' ', nivel) + nome_fun as organograma,
215     nivel,
216     salario,
217     caminho
218 from hierarquia_rh
219 order by caminho;
220
```

organograma	nivel	salario	caminho
1 Fernando Costa	0	19000.00	Fernando Costa
2 Ana Paula Ribeiro	1	11000.00	Fernando Costa > Ana Paula Ribeiro
3 Beatriz Santos	2	4800.00	Fernando Costa > Ana Paula Ribeiro > Beatriz Santos
4 Larissa Gomes	2	3200.00	Fernando Costa > Ana Paula Ribeiro > Larissa Gomes
5 Priscila Cardoso	2	2800.00	Fernando Costa > Ana Paula Ribeiro > Priscila Card...
6 Rafael Lima	2	5300.00	Fernando Costa > Ana Paula Ribeiro > Rafael Lima
7 Roberto Almeida	0	3200.00	Roberto Almeida
8 Carlos Mendes	1	19765.35	Roberto Almeida > Carlos Mendes

3.3. EXISTS — Verificação de Funcionários com Benefícios Ativos

Para verificar quais funcionários possuem ao menos um benefício ativo cadastrado, foi utilizado o operador EXISTS, que interrompe a busca assim que encontra a primeira ocorrência, tornando a consulta mais eficiente do que o IN para esse tipo de verificação. O resultado retornou 14 funcionários com benefícios ativos, exibindo nome, salário e departamento de cada um.


```

4
5
6
7 /* exemplo 01 - lista os funcionarios que possuem ao menos um beneficio ativo,
8 demonstrando as duas formas equivalentes de fazer essa consulta: exists e in */
9
10
11 select
12     f.id_funcionario,
13     f.nome_fun,
14     f.salario,
15     d.nome_dep
16 from funcionario f
17 inner join departamento d on d.id_departamento = f.id_departamento
18 where exists (
19     select 1
20     from beneficio b
21     where b.id_funcionario = f.id_funcionario
22     and b.data_fim is null
23 )
24 and f.data_demissao is null
25 order by f.nome_fun
26
27 /* exemplo 02 - lista os funcionarios ativos que ainda nao possuem nenhuma avaliacao
28 de desempenho registrada, ordenando pelos mais antigos na empresa. */
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

id_funcionario	nome_fun	salario	nome_dep
1	Ana Paula Ribeiro	11000.00	Recursos Humanos
2	Beatriz Santos	4800.00	Recursos Humanos
3	Camila Rocha	8500.00	Financeiro
4	Carlos Mendes	19765.35	Tecnologia da Informacao
5	Diego Nascimento	9000.00	Financeiro
6	Fernanda Costa	19000.00	Recursos Humanos
7	Juliana Ferreira	7000.00	Tecnologia da Informacao
8	Larissa Gomes	3200.00	Recursos Humanos
9	Leonardo Pereira	12000.00	Juridico
10	Pedro Oliveira	10541.52	Tecnologia da Informacao
11	Rafael Lima	5300.00	Recursos Humanos
12	Roberto Almeida	32210.20	Tecnologia da Informacao
13	Tatiane Nunes	7200.00	Operacoes
14	Thiago Barbosa	8491.78	Tecnologia da Informacao

Consulta executada com êxito. (local)\SQLSERVER (16.0 RTM) CAROL-LT\Carol (81) sistema_rh 00:00:00 Linha: 1, Col: 1 14 linhas

3.4. IN — Filtro de Funcionários por Nível de Cargo

Para listar os funcionários de nível Sênior e Pleno, foi utilizado o operador IN com uma lista de valores, filtrando diretamente pelo campo nivel_cargo. O resultado retornou 18 colaboradores ordenados por nível e salário decrescente, permitindo ao RH analisar a distribuição salarial entre as senioridades mais experientes da empresa.

```

444
445
446
447 /* exemplo 03 - lista os funcionarios de nivel senior e pleno, ordenando por nivel
448 e salario para facilitar analises de faixa salarial por senioridade. */
449
450
451 select
452     f.nome_fun,
453     f.salario,
454     c.nome_cargo,
455     c.nivel_cargo,
456     d.nome_dep
457 from funcionario f
458 inner join cargo c on c.id_cargo = f.id_cargo
459 inner join departamento d on d.id_departamento = f.id_departamento
460 where c.nivel_cargo in ('Senior', 'Pleno')
461 and f.data_demissao is null
462 order by c.nivel_cargo, f.salario desc
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600

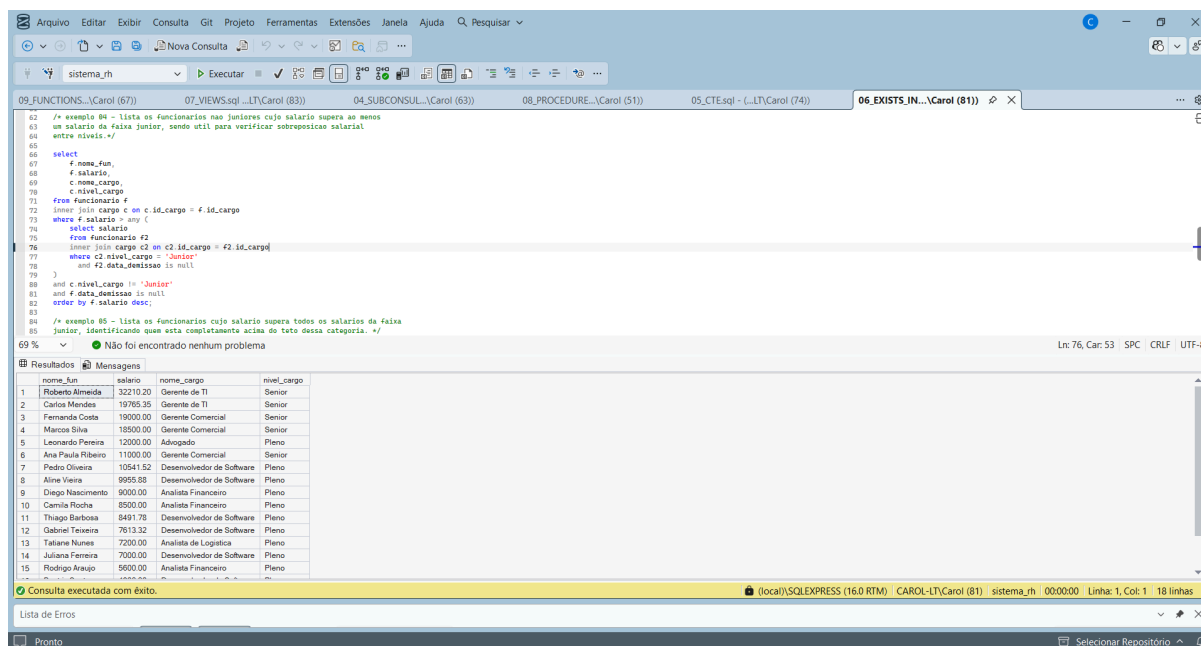
```

nome_fun	salario	nome_cargo	nivel_cargo	nome_dep
1	Leonardo Pereira	Advogado	Pleno	Juridico
2	Pedro Oliveira	Desenvolvedor de Software	Pleno	Tecnologia da Informacao
3	Aline Vieira	Desenvolvedor de Software	Pleno	Tecnologia da Informacao
4	Diego Nascimento	Analista Financeiro	Pleno	Financeiro
5	Camila Rocha	Analista Financeiro	Pleno	Financeiro
6	Thiago Barbosa	Desenvolvedor de Software	Pleno	Tecnologia da Informacao
7	Gabriel Teixeira	Desenvolvedor de Software	Pleno	Tecnologia da Informacao
8	Tatiane Nunes	Analista de Logistica	Pleno	Operacoes
9	Juliana Ferreira	Desenvolvedor de Software	Pleno	Tecnologia da Informacao
10	Rodrigo Araujo	Analista Financeiro	Pleno	Financeiro
11	Beatriz Santos	Desenvolvedor de Software	Pleno	Recursos Humanos
12	Edson Batista	Analista de Logistica	Pleno	Logistica
13	Monica Ramos	Analista de Logistica	Pleno	Atendimento ao Cliente
14	Roberto Almeida	Gerente de TI	Senior	Tecnologia da Informacao
15	Carlos Mendes	Gerente de TI	Senior	Tecnologia da Informacao

Consulta executada com êxito. (local)\SQLSERVER (16.0 RTM) CAROL-LT\Carol (81) sistema_rh 00:00:00 Linha: 1, Col: 1 18 linhas

3.5. ANY — Comparação Salarial com a Faixa Júnior

Para identificar funcionários não juniores cujo salário supera ao menos um salário da faixa júnior, foi utilizado o operador ANY. Essa consulta é útil para detectar sobreposição salarial entre níveis de cargo, evidenciando casos onde um funcionário Pleno ou Sênior recebe apenas um pouco acima do teto júnior. O resultado retornou 18 funcionários.



```
62 /* exemplo 84 - lista os funcionarios nao juniores cujo salario supera ao menos
63 um salario da faixa junior, sendo util para verificar sobreposicao salarial
64 entre niveis: */
65
66 select
67     f.nome_fun,
68     f.salario,
69     c.nome_cargo,
70     c.nivel_cargo
71 from funcionario f
72 inner join cargo c on c.id_cargo = f.id_cargo
73 where f.salario >= any (
74     select salario
75     from funcionario f2
76     inner join cargo c2 on c2.id_cargo = f2.id_cargo
77     where c2.nivel_cargo = 'Junior'
78     and f2.data_demissao is null
79 )
80 and c.nivel_cargo != 'Junior'
81 and f.data_demissao is null
82 order by f.salario desc;
83
84 /* exemplo 85 - lista os funcionarios cujo salario supera todos os salarios da faixa
85 junior, identificando quem esta completamente acima do teto dessa categoria. */
```

69 % Não foi encontrado nenhum problema

nome_fun	salario	nome_cargo	nivel_cargo
1 Roberto Almeida	32210.20	Gerente de TI	Senior
2 Carlos Mendes	19765.35	Gerente de TI	Senior
3 Fernanda Costa	19000.00	Gerente Comercial	Senior
4 Marcos Silva	18500.00	Gerente Comercial	Senior
5 Leonardo Pereira	12000.00	Advogado	Pleno
6 Ana Paula Ribeiro	11000.00	Gerente Comercial	Senior
7 Pedro Oliveira	10541.52	Desenvolvedor de Software	Pleno
8 Alire Vieira	9995.68	Desenvolvedor de Software	Pleno
9 Diego Nascimento	9000.00	Analista Financeiro	Pleno
10 Camila Rocha	8500.00	Analista Financeiro	Pleno
11 Thiago Barbosa	8491.78	Desenvolvedor de Software	Pleno
12 Gabriel Teixeira	7613.32	Desenvolvedor de Software	Pleno
13 Taliana Nunes	7200.00	Analista de Logística	Pleno
14 Juliana Ferreira	7000.00	Desenvolvedor de Software	Pleno
15 Rodrigo Araujo	5600.00	Analista Financeiro	Pleno

Consulta executada com êxito.

3.6. ALL — Funcionários Completamente Acima da Faixa Júnior

Para identificar os funcionários cujo salário supera todos os salários da faixa júnior, foi utilizado o operador ALL, que só retorna verdadeiro quando a condição é satisfeita para todos os valores da subconsulta. O resultado retornou 15 funcionários que estão completamente acima do teto salarial júnior, incluindo todos os Seniores e parte dos Plenos.

```

63
64 /* exemplo 88 - lista os funcionarios cujo salario supera todos os salarios da faixa
65 Junior, identificando quem esta completamente acima do teto dessa categoria. */
66
67 select
68     f.nome_fun,
69     f.salario,
70     c.nome_cargo,
71     c.nivel_cargo,
72     d.nome_dep
73 from funcionario f
74 inner join cargo c on c.id_cargo = f.id_cargo
75 inner join departamento d on d.id_departamento = f.id_departamento
76 where f.salario > all (
77     select salario
78     from funcionario f2
79     inner join cargo c2 on c2.id_cargo = f2.id_cargo
80     where c2.nivel_cargo = 'Junior'
81     and f2.data_demissao is null
82 )
83 and f.data_demissao is null
84 order by f.salario desc;

```

nome_fun	salario	nome_cargo	nivel_cargo	nome_dep
1 Roberto Almeida	32210.20	Gerente de TI	Senior	Tecnologia da Informacao
2 Carlos Mendes	19765.35	Gerente de TI	Senior	Tecnologia da Informacao
3 Fernanda Costa	19000.00	Gerente Comercial	Senior	Recursos Humanos
4 Marcos Silva	18500.00	Gerente Comercial	Senior	Comercial
5 Leonardo Pereira	12000.00	Advogado	Pleno	Juridico
6 Ana Paula Ribeiro	11000.00	Gerente Comercial	Senior	Recursos Humanos
7 Pedro Oliveira	10541.52	Desenvolvedor de Software	Pleno	Tecnologia da Informacao
8 Aline Vieira	9555.88	Desenvolvedor de Software	Pleno	Tecnologia da Informacao
9 Diego Nascimento	9000.00	Analista Financeiro	Pleno	Financeiro
10 Camila Rocha	8500.00	Analista Financeiro	Pleno	Financeiro
11 Thiago Barbosa	8491.78	Desenvolvedor de Software	Pleno	Tecnologia da Informacao
12 Gabriel Teixeira	7613.32	Desenvolvedor de Software	Pleno	Tecnologia da Informacao
13 Tatiane Nunes	7200.00	Analista de Logistica	Pleno	Operacoes
14 Juliana Ferreira	7000.00	Desenvolvedor de Software	Pleno	Tecnologia da Informacao
15 Rodrigo Araujo	5600.00	Analista Financeiro	Pleno	Financeiro

3.7. VIEW — Indicadores Gerenciais por Departamento

Foi criada a view `vw_indicadores_departamento` para consolidar os principais indicadores de pessoal por departamento, incluindo total de funcionários, folha salarial total, média salarial, menor e maior salário e média de avaliação de desempenho do último período fechado. Essa view permite ao RH e à diretoria consultar o panorama completo de cada área sem precisar construir a consulta do zero a cada acesso. O resultado exibe os 10 departamentos cadastrados com seus respectivos indicadores.

```

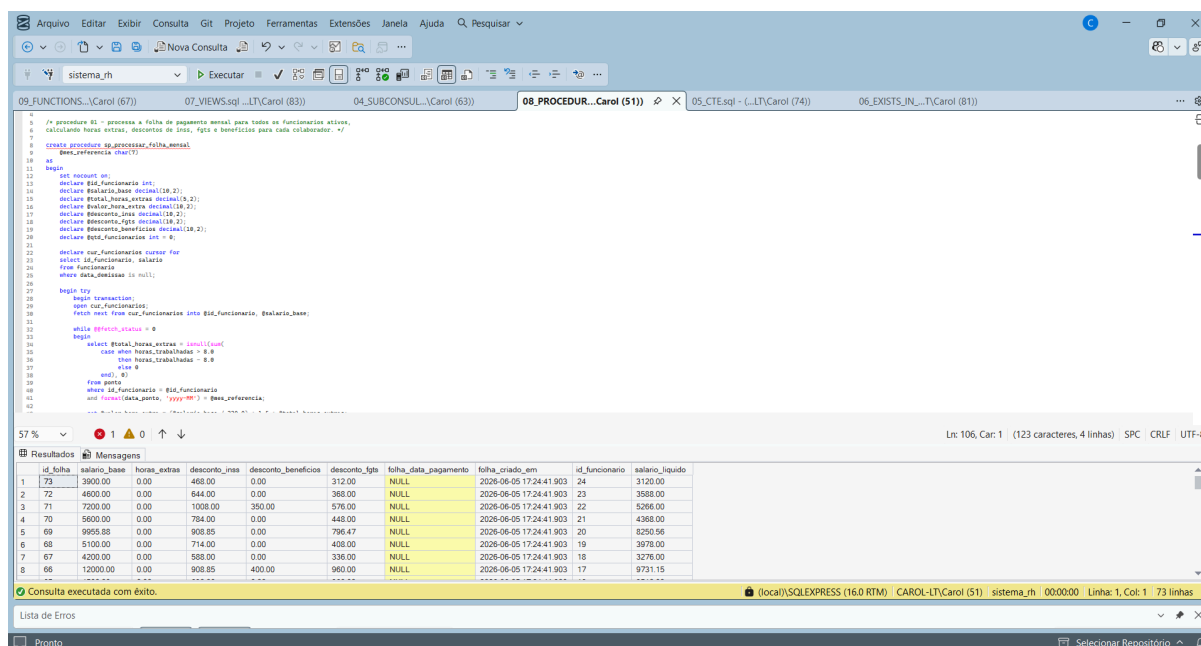
60
61 /* view 83 - agrupa por departamento os principais indicadores de pessoal, como total
62 de funcionarios, folha salarial e media de avaliacao do ultimo periodo fechado. */
63 create view vw_indicadores_departamento as
64 select
65     d.id_departamento,
66     d.nome_dep,
67     d.ativo_dep,
68     cast((f.salario) as decimal(18,2)) as total_funcionarios,
69     cast((f.salario) as decimal(18,2)) as total_salario,
70     cast((f.salario) as decimal(18,2)) as media_salario,
71     cast((f.salario) as decimal(18,2)) as menor_salario,
72     cast((f.salario) as decimal(18,2)) as maior_salario,
73     cast((f.salario) as decimal(18,2)) as media_avaliacao
74 from departamento d
75 left join funcionario f on f.id_departamento = d.id_departamento
76 left join avaliacao_desempenho av on av.id_funcionario = f.id_funcionario
77 and av.periodo = '2023-12-31'
78 group by
79     d.id_departamento,
80     d.nome_dep,
81     d.ativo_dep;

```

id_departamento	nome_dep	ativo_dep	total_funcionarios	folha_salario_total	media_salario	menor_salario	maior_salario	media_avaliacao
1	Tecnologia da Informacao	S	7.00	95578.05	13654.01	7000.00	32210.20	8.70
2	Recursos Humanos	S	6.00	46100.00	7683.33	2800.00	19000.00	6.50
3	Financeiro	S	3.00	23100.00	7700.00	5600.00	9000.00	8.33
4	Comercial	S	4.00	32300.00	8075.00	4200.00	18500.00	7.53
5	Marketing	S	0.00	NULL	NULL	NULL	NULL	NULL
6	Operacoes	S	1.00	7200.00	7200.00	7200.00	7200.00	8.50
7	Juridico	S	1.00	12000.00	12000.00	12000.00	12000.00	9.50
8	Logistica	S	1.00	4600.00	4600.00	4600.00	4600.00	6.00
9	Desenvolvimento ao Cliente	S	1.00	2900.00	2900.00	2900.00	2900.00	7.50
10	Administrativo	N	0.00	NULL	NULL	NULL	NULL	NULL

3.8. Procedure — Processamento da Folha de Pagamento Mensal

Foi criada a procedure `sp_processar_folha_mensal` para automatizar o processamento da folha de pagamento, percorrendo todos os funcionários ativos por meio de um cursor e calculando para cada um as horas extras, os descontos de INSS por faixa progressiva, o FGTS e os descontos de benefícios ativos, inserindo os registros na tabela `folha_pagamento` dentro de uma transação com tratamento de erros. A execução processou 73 registros com sucesso para o mês de referência informado.



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows the code for the stored procedure `sp_processar_folha_mensal` in the `08_PROCEDUR...Carol (51)` database. The procedure is designed to process monthly payroll for active employees, calculating overtime, INSS (Social Security) discounts, FGTS (Funding for Guaranteed Retirement Savings) discounts, and active benefits. It uses a cursor to iterate through employees and inserts the calculated data into the `folha_pagamento` table within a transaction, handling errors.

The bottom pane shows the results of the procedure execution. A message indicates "Consulta executada com êxito." (Query executed successfully). Below the message, a table displays the results for 73 employees. The table has columns for employee ID, salary base, overtime hours, INSS discount, benefits discount, FGTS discount, payment date, creation date, employee ID, and liquid salary.

id_folha	salario_base	horas_extras	desconto_inss	desconto_beneficios	desconto_fgts	folha_data_pagamento	folha_criado_em	id_funcionario	salario_liquido
73	3900.00	0.00	468.00	0.00	312.00	NULL	2026-06-05 17:24:41.903	24	3120.00
72	4000.00	0.00	480.00	0.00	360.00	NULL	2026-06-05 17:24:41.903	23	3568.00
71	7200.00	0.00	1008.00	350.00	576.00	NULL	2026-06-05 17:24:41.903	22	5266.00
70	5600.00	0.00	784.00	0.00	448.00	NULL	2026-06-05 17:24:41.903	21	4368.00
69	9955.88	0.00	908.85	0.00	796.47	NULL	2026-06-05 17:24:41.903	20	8250.56
68	5100.00	0.00	714.00	0.00	408.00	NULL	2026-06-05 17:24:41.903	19	3978.00
67	4200.00	0.00	508.00	0.00	336.00	NULL	2026-06-05 17:24:41.903	18	3276.00
66	12000.00	0.00	908.85	400.00	960.00	NULL	2026-06-05 17:24:41.903	17	9731.15

3.9. Function — Cálculo do Salário Líquido

Foi criada a scalar function `fn_calcular_salario_liquido` para calcular o salário líquido de qualquer funcionário descontando INSS, FGTS e benefícios ativos, podendo ser chamada diretamente em uma consulta `SELECT`. O resultado exibe para cada funcionário o salário bruto, o salário líquido calculado e o total de descontos, com Roberto Almeida apresentando o maior salário bruto de R\$ 32.210,20 e salário líquido de R\$ 27.324,53.

```
09_FUNCTIONS...Carol (67) | 07_VIEWS.sql...LT\Carol (83) | 04_SUBCONSUL...Carol (63) | 08_PROCEDURE...Carol (51) | 05_CTE.sql - (...LT\Carol (74) | 06_EXISTS_IN...T\Carol (81) | 10_TRIGGERS...Carol (52) |
```

```
1 use sistema_rh;
2
3 /* Função #1 - calcula o salario liquido de um funcionario descontando imps, fgts
4 e beneficios sociais, retornando null caso o funcionario nao seja encontrado */
5
6 create function fn_calcular_salario_liquido (
7     @id_funcionario int
8 )
9 returns decimal(18,2)
10 as
11 begin
12     declare @salario_bruto decimal(18,2);
13     declare @desconto_imps decimal(18,2);
14     declare @desconto_fgts decimal(18,2);
15     declare @desconto_beneficios decimal(18,2);
16     declare @salario_liquido decimal(18,2);
17
18     select @salario_bruto = salario
19     from funcionario
20     where id_funcionario = @id_funcionario;
21
22     if @salario_bruto is null
23     return null;
24
25     set @desconto_imps =
26     case
27         when @salario_bruto <= 1412.00 then @salario_bruto * 0.075
28         when @salario_bruto > 1412.00 then @salario_bruto * 0.09
29         when @salario_bruto > 2824.00 then @salario_bruto * 0.12
30         when @salario_bruto > 5648.00 then @salario_bruto * 0.14
31         else 0.00
32     end;
33
34     set @desconto_fgts = @salario_bruto * 0.08;
35
36     select @desconto_beneficios = sum(cas(c.valor_mensal(), 0))
37     from beneficios
38     where id_funcionario = @id_funcionario
39     and data_fim is null;
40
41     set @salario_liquido = @salario_bruto - @desconto_imps - @desconto_fgts - @desconto_beneficios;
42
43     return @salario_liquido;
44 end;
```

	nome_fun	salario_bruto	salario_liquido_calculado	total_descontos
1	Roberto Almeida	32210.20	27324.53	4885.67
2	Carlos Mendes	19765.35	16475.27	3290.08
3	Fernanda Costa	19000.00	15171.15	3828.85
4	Marcos Silva	10500.00	10111.15	2388.85
5	Leonardo Pereira	12000.00	9731.15	2268.85
6	Ana Paula Ribeiro	11000.00	8111.15	2888.85
7	Elaine Oliveira	10641.67	8130.36	2511.31

Consultas executadas com sucesso. (local)\SQLEXPRESS (16.0 RTM) - CAROL-LT\Carol (67) - sistema_rh 00:00:00 Linhas: 1, Col: 1 24 linhas

3.10. Trigger AFTER — Auditoria de Alterações Salariais

Foi criada a trigger `trg_auditoria_salario` que é disparada automaticamente após qualquer UPDATE na coluna `salario` da tabela `funcionario`, registrando na tabela `log_funcionario` o valor anterior, o valor novo, a data da operação e o usuário responsável. O log gerado comprova o rastreamento completo das alterações, registrando os reajustes aplicados com data, hora e usuário do banco de dados.

```
09_FUNCTIONS...Carol (67) | 07_VIEWS.sql...LT\Carol (83) | 08_PROCEDURE...Carol (51) | 05_CTE.sql - (...LT\Carol (74) | 06_EXISTS_IN...T\Carol (81) | 10_TRIGGERS...Carol (52) |
```

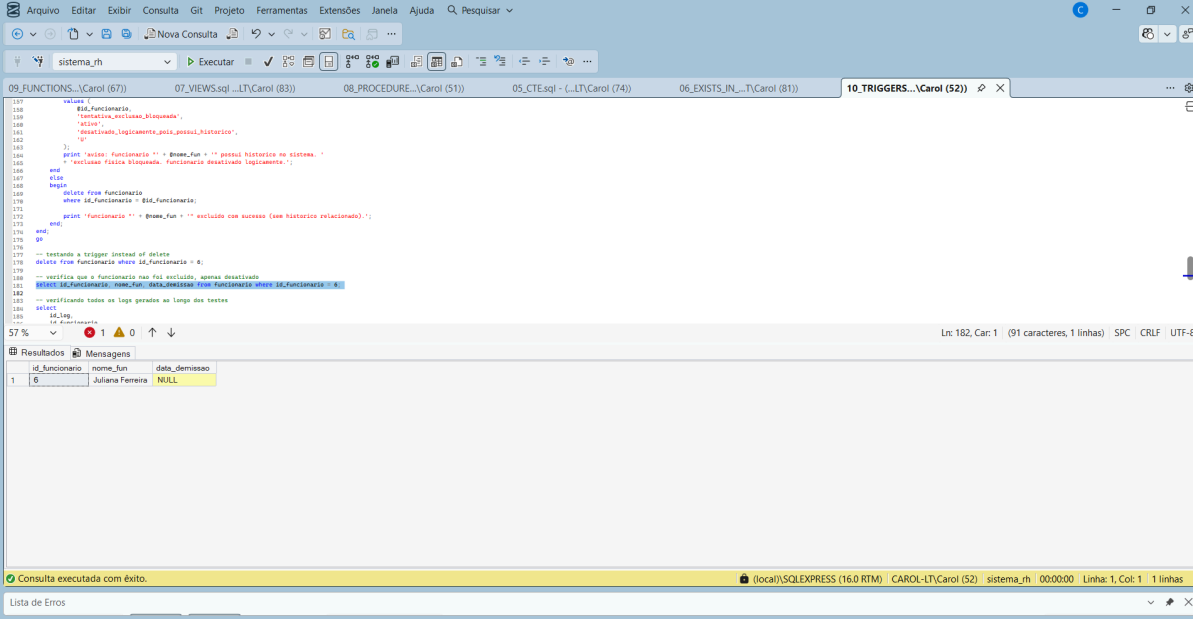
```
1 create trigger trg_auditoria_salario
2 on funcionario after update as
3
4 begin
5     set nocount on;
6     if @@triggerschained = 1
7     begin
8         insert into log_funcionario (
9             id_funcionario,
10             campo_alterado,
11             valor_anterior,
12             valor_novo,
13             operacao,
14             data_operacao,
15             usuario_bd
16         )
17         select
18             @id_funcionario,
19             'salario',
20             cast(old_salary as varchar(50)),
21             cast(new_salary as varchar(50)),
22             'U',
23             getdate(),
24             system_user
25         from inserted i
26         inner join deleted d on d.id_funcionario = i.id_funcionario
27         where i.salary <> d.salary;
28     end;
29 end;
```

	id_log	id_funcionario	campo_alterado	valor_anterior	valor_novo	operacao	data_operacao	usuario_bd
1	31	8	id_cargo	2	1	U	2026-06-05 16:47:31.287	CAROL-LT\Carol
2	29	6	salario	9516.65	7000.00	U	2026-06-05 16:41:57.900	CAROL-LT\Carol
3	22	1	salario	29282.00	32210.20	U	2026-06-05 16:33:45.047	CAROL-LT\Carol
4	23	3	salario	17968.50	19765.35	U	2026-06-05 16:33:45.047	CAROL-LT\Carol
5	24	6	salario	8651.50	9516.65	U	2026-06-05 16:33:45.047	CAROL-LT\Carol
6	25	7	salario	9583.20	10541.52	U	2026-06-05 16:33:45.047	CAROL-LT\Carol
7	26	13	salario	7719.80	8491.78	U	2026-06-05 16:33:45.047	CAROL-LT\Carol
8	27	15	salario	6921.20	7613.32	U	2026-06-05 16:33:45.047	CAROL-LT\Carol
9	28	20	salario	9050.80	9955.88	U	2026-06-05 16:33:45.047	CAROL-LT\Carol
10	15	1	salario	26620.00	29282.00	U	2026-06-05 16:33:18.180	CAROL-LT\Carol
11	16	3	salario	16335.00	17968.50	U	2026-06-05 16:33:18.180	CAROL-LT\Carol
12	17	6	salario	7865.00	8651.50	U	2026-06-05 16:33:18.180	CAROL-LT\Carol
13	18	7	salario	8712.00	9583.20	U	2026-06-05 16:33:18.180	CAROL-LT\Carol
14	19	13	salario	7018.00	7719.80	U	2026-06-05 16:33:18.180	CAROL-LT\Carol
15	20	15	salario	6292.00	6921.20	U	2026-06-05 16:33:18.180	CAROL-LT\Carol

Consultas executadas com sucesso. (local)\SQLEXPRESS (16.0 RTM) - CAROL-LT\Carol (52) - sistema_rh 00:00:00 Linhas: 1, Col: 1 16 linhas

3.11. Trigger INSTEAD OF — Bloqueio de Exclusão Física

Foi criada a trigger `trg_bloquear_exclusao_funcionario` que intercepta qualquer tentativa de DELETE na tabela `funcionario`. Quando o funcionário possui histórico em folha de pagamento, ponto ou avaliações, a exclusão física é bloqueada e substituída pela desativação lógica via preenchimento da `data_demissao`, preservando a integridade do histórico. A verificação confirma que a funcionária Juliana Ferreira permanece na tabela com `data_demissao` nula, pois a trigger bloqueou a exclusão com sucesso.



The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays the script for the trigger `trg_bloquear_exclusao_funcionario` in the `10_TRIGGERS...Carol (52)` database. The script is a T-SQL `CREATE TRIGGER` statement that intercepts `DELETE` operations on the `funcionario` table. It checks for related records in `funcionario_historico`, `funcionario_ponto`, and `funcionario_avaliacao` tables. If any related records exist, it prints a message and sets `data_demissao` to `NULL` instead of deleting the record. The bottom pane shows the execution results of the script, which is a single row with the following data:

id_funcionario	nome_fun	data_demissao
1	Juliana Ferreira	NULL

The status bar at the bottom indicates that the query was executed successfully.

4 ESTUDO DE CASO

O setor de Recursos Humanos (RH) é responsável pela administração das informações relacionadas aos colaboradores de uma organização, desempenhando um papel fundamental na gestão de pessoas e no cumprimento das obrigações trabalhistas. Entre os principais dados gerenciados pelo RH estão os cadastros de funcionários, cargos, departamentos, registros de jornada de trabalho, folha de pagamento, benefícios e avaliações de desempenho. Essas informações possuem forte dependência entre si, uma vez que o cálculo da folha de pagamento depende dos registros de ponto, os registros de ponto estão vinculados aos funcionários, e cada funcionário está associado a um cargo e a um departamento. Dessa forma, a correta organização e integração desses dados é essencial para garantir a eficiência e a confiabilidade dos processos administrativos.

Quando as informações do setor de Recursos Humanos são armazenadas em planilhas ou em sistemas sem uma estrutura adequada de banco de dados, diversos problemas podem surgir. É comum ocorrer a duplicação de dados, com o mesmo funcionário sendo registrado em diferentes arquivos ou abas, aumentando o risco de inconsistências. Além disso, alterações realizadas em uma informação, como a atualização de um salário, podem não ser refletidas em todos os locais onde o dado está armazenado. A ausência de mecanismos de controle também dificulta a manutenção do histórico das operações e a realização de auditorias, comprometendo a rastreabilidade das informações. Outro desafio é a dificuldade de relacionar dados de diferentes processos, como cruzar registros de ponto com informações da folha de pagamento, tornando as atividades mais lentas e suscetíveis a erros.

Os bancos de dados relacionais oferecem mecanismos que solucionam grande parte desses problemas ao garantir a integridade, a consistência e a segurança das informações. Por meio das chaves estrangeiras, é possível assegurar a integridade referencial entre as tabelas, impedindo, por exemplo, o registro de um ponto para um funcionário inexistente. As restrições de integridade (constraints) permitem a implementação de regras de negócio diretamente no banco de dados, garantindo

que valores inválidos não sejam armazenados, como salários negativos ou notas de avaliação fora da faixa permitida. Além disso, gatilhos (triggers) podem automatizar processos de auditoria e registro de alterações, enquanto procedimentos armazenados (stored procedures) reduzem a ocorrência de erros humanos em operações críticas, como o fechamento da folha de pagamento. Segundo os conceitos apresentados por autores como Ramez Elmasri, Shamkant Navathe e Abraham Silberschatz, a utilização de bancos de dados relacionais é fundamental para garantir a integridade e a confiabilidade das informações corporativas.

Neste trabalho foi desenvolvido um modelo de banco de dados relacional voltado ao contexto de Recursos Humanos, composto por oito tabelas relacionadas que abrangem os principais módulos de gestão de pessoas, incluindo cadastro de colaboradores, cargos, departamentos, controle de ponto, folha de pagamento, benefícios, avaliações de desempenho e auditoria. Sobre essa estrutura foram aplicados diversos recursos da linguagem SQL e dos Sistemas Gerenciadores de Banco de Dados, como subconsultas, Expressões de Tabela Comuns (CTEs), visões (views), procedimentos armazenados (stored procedures), funções (functions) e gatilhos (triggers). Esses recursos foram utilizados para implementar regras de negócio, automatizar processos e fornecer mecanismos de consulta e controle capazes de atender às necessidades reais de gerenciamento de informações em um setor de Recursos Humanos.

5 CONCLUSÃO

O desenvolvimento deste trabalho permitiu compreender a importância dos bancos de dados relacionais na gestão de informações do setor de Recursos Humanos. A modelagem proposta demonstrou como dados relacionados a funcionários, cargos, departamentos, controle de ponto e folha de pagamento podem ser organizados de forma estruturada, garantindo maior segurança, consistência e eficiência no armazenamento e na manipulação das informações.

Além da modelagem relacional, a utilização de recursos oferecidos pelos Sistemas Gerenciadores de Banco de Dados, como gatilhos (triggers), procedimentos armazenados (stored procedures) e visões (views), evidenciou a capacidade de automatizar processos, reduzir erros operacionais e fortalecer a integridade dos dados. Essas funcionalidades contribuem para que as regras de negócio sejam aplicadas diretamente no banco de dados, proporcionando maior confiabilidade às operações realizadas pelo setor.

Dessa forma, conclui-se que a aplicação de um banco de dados relacional no contexto de Recursos Humanos representa uma solução eficiente para o gerenciamento das informações organizacionais, oferecendo suporte às atividades administrativas e auxiliando na tomada de decisões. O trabalho também reforça a relevância dos conceitos de modelagem e administração de bancos de dados para o desenvolvimento de sistemas capazes de atender às necessidades reais das empresas.

6 REFERÊNCIAS BIBLIOGRÁFICAS

DATE, C. J. Introdução a sistemas de bancos de dados. 8. ed. Rio de Janeiro: Elsevier, 2004.

ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de banco de dados. 7. ed. São Paulo: Pearson, 2019.

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. Sistema de banco de dados. 7. ed. Rio de Janeiro: LTC, 2020.

MICROSOFT. Subqueries (SQL Server). Disponível em: <https://learn.microsoft.com/pt-br/sql/relational-databases/performance/subqueries>. Acesso em: 4 jun. 2026.

MICROSOFT. WITH common_table_expression (Transact-SQL). Disponível em: <https://learn.microsoft.com/pt-br/sql/t-sql/queries/with-common-table-expression-transact-sql>. Acesso em: 4 jun. 2026.

MICROSOFT. EXISTS (Transact-SQL). Disponível em: <https://learn.microsoft.com/pt-br/sql/t-sql/language-elements/exists-transact-sql>. Acesso em: 4 jun. 2026.

MICROSOFT. Views. Disponível em: <https://learn.microsoft.com/pt-br/sql/relational-databases/views/views>. Acesso em: 4 jun. 2026.

MICROSOFT. CREATE VIEW (Transact-SQL). Disponível em: <https://learn.microsoft.com/pt-br/sql/t-sql/statements/create-view-transact-sql>. Acesso em: 4 jun. 2026.

MICROSOFT. Stored procedures (Database Engine). Disponível em: <https://learn.microsoft.com/pt-br/sql/relational-databases/stored-procedures/stored-procedures-database-engine>. Acesso em: 4 jun. 2026.

MICROSOFT. User-defined functions. Disponível em: <https://learn.microsoft.com/pt-br/sql/relational-databases/user-defined-functions/user-defined-functions>. Acesso em: 4 jun. 2026.

MICROSOFT. DML triggers. Disponível em: <https://learn.microsoft.com/pt-br/sql/relational-databases/triggers/dml-triggers>. Acesso em: 4 jun. 2026.

